Merging IFC, USD, and ECS

A strategic approach to integrating the semantic business logic of Industry Foundation Classes within the high-performance architecture of Universal Scene Description using an Entity-Component-System paradigm.

The Strategy

- The Industry Foundation Classes (IFC) schema excels at defining the strict semantic logic, relationships, and property data of buildings.
- Universal Scene Description (USD) provides an unparalleled, high-performance framework for composing massive 3D scenes.
- The emergent **IFC-ECS Prototype** introduces a modernized approach to break down monolithic IFC4.x instances into flattened entities and components.
- By migrating IFC-ECS structures into USD native schemas, we seamlessly bridge standard BIM data with advanced rendering, physics simulation, and real-time digital twins.

Core USD Concepts

- **class:** Establishes a reusable template. In IFC, this directly maps to abstract base definitions or `IfcTypeObject` elements, defining pure structural topology without a physical presence.
- **def:** Defines a concrete, physical instance. In IFC, this correlates to an `IfcProduct` occurrence instantiated within a specific level of the spatial hierarchy.
- **over:** An overriding declaration that retroactively augments an object. This is how we inject IFC semantics, specific property sets, and massive geometric arrays.

Mapping USD to the IFC-ECS Prototype

- **`def` = Entity:** In USD, this creates a "Prim". In the IFC-ECS model, this maps to a first-class object defined by an `entityGuid` (derived from the IFC GlobalId).
- **`over` = Component:** In USD, this attaches modular data. In IFC-ECS, this maps perfectly to the generated JSON components (e.g., an `IfcPropertySetComponent` possessing a unique `componentGuid`) that reference the parent `entityGuid`.
- **`class` = Prefab / Archetype:** Defines a reusable blueprint. An Entity inherits this to instantly receive all pre-packaged topological components.
- **System = External Engines:** USD and the IFC-ECS server act as the database. External applications query the REST API or USD graph to

execute logic.

Interoperability: IFC/USD & IFC-ECS

- **Shared Architectural DNA:** Both frameworks abandon deep Object-Oriented hierarchies in favor of composition. USD's composition arcs perfectly mirror the IFC-ECS prototype's flattened JSON structure.
- **Unified Identity Mapping:** The IFC-ECS prototype elevates IFC GlobalIds to `entityGuid`s. USD can utilize these exact same GUIDs for its `def` prim names, establishing a robust 1:1 cross-platform link.
- **Granular Data Syncing:** Because IFC-ECS separates data into discrete components (e.g., isolating `IfcPropertySetComponent`), a USD pipeline can selectively query a REST API component server to inject specific semantic payloads via `over` without loading massive files.
- **Decoupled Geometry & Logic:** IFC-ECS strips complex parametric representations into triangulated `IfcShapeRepresentationComponent`s.

These can be seamlessly piped into USD as `UsdGeom:Mesh` components while preserving the abstract entity logic.

Understanding "class"

In USD, a `class` establishes a reusable blueprint without actually placing an object into the scene hierarchy.

In the ECS paradigm, this acts as the **Prefab or Archetype**. You define it once, and instantiate it many times.

In our IFC merging strategy, classes are leveraged to define the foundational definition of a building element—such as a generic Wall. This includes mapping out its necessary sub-components, such as its `Body` (Mesh), `Axis`, and `Directrix` (BasisCurves).

Understanding "def"

A `def` block creates a tangible, concrete instance in the USD scene. It moves a concept from a blueprint into physical space.

In the ECS paradigm, this acts as the **Entity**—a unique identifier or container that exists in the world, ready to hold specific data components.

For our IFC model, this is the exact moment an abstract wall "class" becomes a physical occurrence. It is instantiated within a strict spatial container, mimicking the IFC Spatial Structure (e.g., inside a specific `IfcSpace` or `IfcBuildingStorey`).

Understanding "over"

The `over` concept is the linchpin for merging IFC's heavy business logic with USD's lightweight scene structure.

Instead of cluttering the initial definition, USD allows us to non-destructively "override" or augment an existing `class` or `def` retroactively.

In the ECS paradigm, this is how we attach **Data Components** to an Entity through composition. We use `over` declarations strictly to inject the semantic payload: designating the `ifc5:class`, assigning custom Property Sets (like external boundaries), linking materials, and streaming raw geometric vertex data.

Understanding "inherits"

In USD, `inherits` is a core composition arc that instructs a prim to derive its characteristics and structure from a base `class`.

In the ECS paradigm, this acts as the **Instantiation Mechanism**. It seamlessly connects a unique physical Entity (the `def`) back to its master Prefab/Archetype (the `class`).

By declaring `"inherits": ["</N9379...>"]`, the concrete instance automatically absorbs all the topological sub-components (like meshes and structural axes) defined in the abstract blueprint, entirely preventing data duplication.

IFC Naming Conventions & Suffixes

- In this schema, you will see base names like `N93791d5d5beb437bb8ec2f1f0ba4bf3b` and concatenated names like `N93791d5d5beb437bb8ec2f1f0ba4bf3b_Body`.
- The **base name** represents the top-level, abstract architectural container. It is the core IFC element (the Entity), such as the overall Wall object, tracked by its `entityGuid`.
- The **concatenated name** represents a specific geometric sub-component belonging to that parent element.
- This structure is necessary because a single IFC Entity might contain multiple distinct representations simultaneously, such as a physical 3D mesh (`_Body`), a centerline (`_Axis`), or a 2D footprint (`_FootPrint`).

- Appending text ensures unique identification for these sub-components within the USD hierarchy without naming collisions.

IFC to USD Pipeline

- **Step 1: class (Prefab)** - Establish the base IFC element templates and required sub-topologies.
- **Step 2: def (Entity)** - Instantiate physical elements precisely within the USD spatial hierarchy using `entityGuid` mapping.
- **Step 3: over (Component: Logic)** - Retroactively inject business logic, IFC classes, and custom JSON property sets fetched from the component server.
- **Step 4: over (Component: Geom)** - Stream massive geometric coordinate arrays (converted to OBJ/Mesh) and complex materials.

Code Summary: Blueprint & Instantiation

1. Blueprint (class / Prefab)

```
{
 "def": "class",
 "type": "UsdGeom:Xform",
 "name": "N93791d..."
}
```

2. Instantiation (def / Entity)

```
{
 "def": "def",
 "name": "Wall",
 "inherits": ["</N93791...>"]
}
```

Code Summary: Augmentation & Naming

3. Augmentation (over / Component)

```
{
 "def": "over",
 "attributes": {
  "ifc5:class": {"code": "IfcWall"}
 }
}
```

4. Name Expansion

While `N9379...` identifies the core Entity container, concatenating a suffix like `_Body` targets a specific geometric Data Component. This allows one abstract IFC Entity to maintain multiple distinct representations (Body, Axis, FootPrint) simultaneously without namespace collisions.

```
{
 "def": "def",
```

```
  "name": "Body",
  "inherits": ["</N9379..._Body>"]
}
```

Example Data Notice

- 2024-11-12 update of the examples.
- (C) buildingSMART International.
- Published under CC BY-ND 4.0.